

# The Bash Tool: Configuration and Security

The bash tool is the workhorse of ragent. It executes shell commands from the agent’s working directory and applies a seven-layer security model.

This document covers:

1. Overview
2. Configuration — the bash block in `ragent.json`
3. The seven-layer security model
4. The `/bash` slash commands
5. Runtime behaviour — timeout, output truncation, process limits
6. YOLO mode
7. Platform notes
8. Reference

---

## Overview

The tool is implemented in `crates/ragent-tools-core/src/bash.rs` (~1750 lines) as the `BashTool` struct. It is a unit struct with no fields of its own — all behaviour is driven by the global `ragent_config` at execution time, populated from your config files.

Every command an agent runs through bash goes through the same pipeline:

`command string`

- |                                                         |                                                                                                        |
|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| 1. Shell discovery<br>(bash / git-bash /<br>powershell) | 2. Seven-layer check<br>(see below)<br><br>approved                                                    |
|                                                         | 3. Process permit (max 16 concurrent)                                                                  |
|                                                         | 4. run via shell (timeout, kill_on_drop)<br>with askpass<br><br>output (truncated<br>to 15k+15k chars) |
-

## Configuration

The bash tool is configured via the top-level bash block in `ragent.json` (global at `~/.config/ragent/ragent.json`, or project at `.ragent/ragent.json`).

### The bash config block

```
{
  "bash": {
    "allowlist": ["curl", "wget"],
    "denylist": ["git push --force", "systemctl disable"]
  }
}
```

| Key       | Type     | Default | Purpose                                                          |
|-----------|----------|---------|------------------------------------------------------------------|
| allowlist | string[] | []      | Command prefixes exempted from the built-in banned-command check |
| denylist  | string[] | []      | Substring patterns that unconditionally reject a command         |

The underlying Rust struct (`crates/ragent-config/src/config.rs`):

```
pub struct BashConfig {
    /// Command prefixes exempted from the banned-command check.
    pub allowlist: Vec<String>,
    /// Patterns that unconditionally reject a command.
    pub denylist: Vec<String>,
}
```

### Merge semantics across global + project config

When both a global and a project config exist, ragent merges them:

- **allowlist** and **denylist** entries are **unioned** (deduplicated). An entry added in either config applies everywhere.

### Config flow

The bash tool does **not** read `Config` directly. At startup (and on `/reload`) ragent copies the merged config into an in-memory snapshot via `BashLists::load_from_config()` (`crates/ragent-config/src/bash_lists.rs`):

- TUI startup: `crates/ragent-tui/src/app/init.rs:397`
- `/reload`: `crates/ragent-tui/src/app/slash.rs:3361`

```
pub fn load_from_config() {
    let lists = match crate::config::Config::load() {
        Ok(cfg) => BashLists {
            allowlist: cfg.bash.allowlist,
            denylist: cfg.bash.denylist,
        },
        ...
    }
```

```
};
}
```

The tool reads from this snapshot at execution time through helpers such as `is_allowlisted()` and `matches_denylist()`.

## The seven-layer security model

Every command is validated by seven layers; all seven must pass. The table below summarises them in the order the checks actually execute, full detail follows.

| Order | Check                      | Source                                                           | Bypassed by YOLO | Bypassed by allowlist |
|-------|----------------------------|------------------------------------------------------------------|------------------|-----------------------|
| 1     | Safe-command whitelist     | SAFE_COMMANDS const                                              | —                | —                     |
| 2     | Banned commands            | BANNED_COMMANDS const                                            | yes              | <b>yes</b>            |
| 3     | Directory-escape           | <code>is_directory_escape_attempt()</code>                       | —                | —                     |
| 4     | Syntax validation          | <code>validate_bash_syntax()</code>                              | —                | —                     |
| 5     | Denied commands & patterns | DENIED_COMMANDS, DENIED_COMMAND_PATTERNS, DENIED_PATTERNS consts | yes              | no                    |
| 6     | User denylist              | <code>matches_denylist()</code>                                  | yes              | no                    |
| 7     | Obfuscation detection      | <code>validate_no_obfuscation()</code>                           | yes              | no                    |

### Layer 1 — Safe-command whitelist

A built-in list of ~50 commands is prefix-matched and auto-approved (it just logs "Safe bash command auto-approved"). A command is safe if it **equals** an entry or **starts with the entry followed by a space**, so `ls` matches `ls -la` and `git` matches `git status`.

```
const SAFE_COMMANDS: &[&str] = &[
    // File management
    "ls", "cd", "pwd", "mkdir", "touch", "cp", "mv",
    // File reading & search
    "cat", "head", "tail", "grep", "egrep", "fgrep", "find", "rg", "wc",
    // Version control
    "git", "gh",
    // Build / package management
    "cargo", "rustc", "rustfmt", "clippy-driver", "npm", "yarn", "pnpm",
    "node", "npx", "python3", "python", "pip", "pip3", "make", "docker-compose",
    // Text / data utilities
    "echo", "printf", "chmod", "jq", "yq", "sed", "awk", "sort", "uniq",
    "cut", "tr", "xargs", "date", "which", "tree", "diff", "patch",
];
```

**Note:** `rm` is deliberately excluded from the whitelist so it always requires a permission decision (and is caught by Layer 3 for destructive forms).

## Layer 2 — Banned commands

A word-boundary match against BANNED\_COMMANDS. These are network / exfiltration and attack tools that are almost never legitimate inside an agent:

```
const BANNED_COMMANDS: &[&str] = &[
    // Network / data-exfiltration tools
    "curl", "wget", "nc", "netcat", "telnet", "axel", "aria2c", "lynx", "w3m",
    // Attack and exploitation tools
    "nmap", "masscan", "nikto", "sqlmap", "hydra", "john", "hashcat",
    "aircrack", "metasploit", "msfconsole", "msfvenom", "burpsuite",
    "ettercap", "arpspoof",
    // Packet capture (enable via YOLO)
    "tcpdump", "wireshark",
];
```

A banned command is rejected **unless**:

- YOLO mode is enabled, **or**
- the command's **first token** is on the user allowlist (this is how you re-enable curl / wget without turning on YOLO).

```
if contains_banned_command(command) {
    if ragent_config::yolo::is_enabled() { /* allow */ }
    else if ragent_config::bash_lists::is_allowlisted(command) { /* allow */ }
    else { bail!("Command rejected: uses banned external tool (curl, wget, nc, ...)"); }
}
```

## Layer 3 — Directory-escape prevention

is\_directory\_escape\_attempt(command, &ctx.working\_dir) rejects commands that cd or pushd out of the working directory via .., ~, \$HOME, \${HOME}, or absolute / paths (and, on Windows, C:\ / \ drive roots). This keeps the agent's filesystem view inside its sandbox. This layer is **never** overridden.

## Layer 4 — Syntax validation

validate\_bash\_syntax() runs the command through the shell's own parser with a **1-second timeout** and kill\_on\_drop(true):

```
bash -n -c "<command>"
```

A non-zero exit produces Bash syntax error: <stderr> and the command is rejected. This uses the actual discovered shell (not a hardcoded sh) so it works on Git Bash too. Syntax validation is **skipped for PowerShell** (which has its own runtime parser and no -n equivalent). This layer is never overridden.

## Layer 5 — Denied commands & patterns

Three sub-checks that are **never** bypassed by the allowlist (only by YOLO):

### 3a. Denied commands (word-boundary):

```
const DENIED_COMMANDS: &[&str] = &[
    "mkfs", "wipefs", // Disk / partition destruction
```

```

    "insmod", // Kernel modifications
    "useradd", "usermod", "groupadd", // User/group manipulation
    "visudo", // System configuration
    "grub-install", "efibootmgr", // Boot/firmware
];

```

### 3b. Denied command patterns (prefix patterns):

```

const DENIED_COMMAND_PATTERNS: &[&str] = &[
    "sudo ", "sudo\t", "su -", "su root", "doas ", // privilege escalation
    "passwd ", // user/group manipulation
    "crontab -", // system configuration
];

```

### 3c. Denied substring patterns — the most destructive forms:

```

const DENIED_PATTERNS: &[&str] = &[
    // Destructive filesystem operations
    "rm -rf /", "rm -r -f /", "rm -fr /", "rm -Rf /", "rmdir /",
    // Disk operations
    "dd if=", "shred /dev",
    // Device writes
    "> /dev/sd", "> /dev/nvme", "> /dev/vd",
    // Fork bomb
    ":{ }{ :|:&};:",
    // Privilege escalation
    "chmod -R 777 /", "chown -R",
    // Network exfiltration of sensitive files
    "curl.*etc/shadow", "wget.*etc/shadow",
    // History / credential file theft
    ".bash_history", ".ssh/id_",
    // Kernel modifications
    "modprobe -r", "sysctl -w",
    // Destructive git operations
    "git push --force", "git push -f ", "git push origin --delete",
    // More destructive patterns
    "rm -rf ~", "rm -rf $HOME", "rm -rf .",
    "chmod 000 /", "chmod -R 000",
    // Data exfiltration via pipes
    "> /dev/tcp", "bash -i >&", "/dev/tcp/", "/dev/udp/",
    "systemctl disable", "systemctl mask", "chattr +i",
];

```

## Layer 6 — User denylist

Substring-matches the command against your configured denylist patterns. This is where project-specific guardrails go. Bypassed in YOLO mode only.

```

if !ragent_config::yolo::is_enabled()
    && let Some(pattern) = ragent_config::bash_lists::matches_denylist(command)
{ bail!("matches user-defined deny pattern ..."); }

```

**Important asymmetry:** an allowlist entry only exempts a command from **Layer 2 (banned commands)**. It does **not** exempt it from the denylist (Layer 6) or the denied-pattern checks (Layer 3). That is by design.

## Layer 7 — Obfuscation detection

`validate_no_obfuscation()` rejects clearly-obfuscated payloads:

- `base64 ... | bash / | sh`
- `python / perl` using `exec( / eval(`
- `$'\xNN` hex-escape sequences
- `eval` combined with `$(...)` command substitution

Bypassed in YOLO mode only.

---

## The /bash slash commands

The TUI exposes interactive management of the allowlist/denylist via `/bash`:

```
/bash add allow <cmd>           # add a command prefix to the allowlist
/bash add deny <pattern>        # add a deny pattern
/bash remove allow <cmd>       # remove an allowlist entry
/bash remove deny <pattern>    # remove a deny pattern
/bash show                     # display current allowlist / denylist
/bash help
```

Each add/remove accepts an optional `--global` flag to target the global config (`~/.config/ragent/agent.json`) instead of the project config (`.agent/agent.json`). Changes persist to disk immediately and update the in-memory snapshot used by the tool.

---

## Runtime behaviour

### Timeout

The default timeout for a shell command is **120 seconds** (`DEFAULT_TIMEOUT_SECS`). It can be overridden per-call via the `timeout` parameter on the tool. On timeout the tool returns "Command timed out after N seconds" with `timed_out: true` metadata.

### Process limits

Before spawning, the tool acquires a process permit from a semaphore that caps **concurrent shell processes at 16** (`crates/agent-types/src/resource.rs`). Combined with a separate cap of 5 concurrent tools, this prevents an agent from forking runaway process trees.

### kill\_on\_drop and process groups

Every process builder sets `kill_on_drop(true)` **and** `process_group(0)`. `kill_on_drop(true)` kills the *direct child* if the command's future is dropped; it does not reach grandchildren. On **timeout**, `ragment` additionally SIGKILLS the entire process group via `kill_process_group(libc::killpg)`, so

descendant processes (e.g. a deadlocked `cargo test` child) are reaped too. Without the timeout path, a plain dropped future kills the direct child only. This closed a real bug fixed in v1.0.60 — repeated bash timeouts previously accumulated orphaned processes. The flags are present at every spawn site (syntax check, all three execute branches, and all background-shell paths).

### Post-exit drain and capture cap

After a command exits, the tool drains remaining pipe output for up to a bounded window (`POST_EXIT_DRAIN_TIMEOUT_SECS = 5 s`) so trailing lines are captured without hanging, and per-stream in-memory capture is capped at `MAX_CAPTURE_BYTES = 2 MiB` to bound memory on chatty processes.

### Output truncation

Output is truncated to the **first 15,000 + last 15,000 characters** (`MAX_OUTPUT = 31,000`) with UTF-8 boundary alignment and an `... lines omitted ...` separator, so very long command output never blows up the agent context.

### Persistent shell state & working directory

The tool maintains a per-session state file that records the current working directory. Commands can `cd` within the sandbox and ragent tracks the change (publishing `Event::ShellCwdChanged` on the event bus) so subsequent commands run from the updated directory.

### Sudo askpass broker

For the rare legitimate `sudo` case, ragent starts an `AskPassBroker` that routes the `SUDO_ASKPASS` prompt through the interactive question dialog instead of hanging on a terminal password prompt.

---

## YOLO mode

YOLO mode (`crates/ragent-config/src/yolo.rs`) is an escape hatch for trusted, single-user environments. It bypasses **Layers 2, 3, 6, and 7** (banned commands, denied commands/patterns, user denylist, and obfuscation detection). Layers 1, 4, and 5 (directory-escape and syntax validation) remain active — YOLO does **not** let the agent escape its working directory or run syntactically-invalid commands.

Enable it with `/yolo`, `Alt+Y`, or by setting `"yolo": true` in config. Once enabled in either the global or project config it stays enabled (OR merge semantics).

---

## Platform notes

- **PowerShell:** syntax pre-validation (Layer 5) is skipped; the runtime parser inside the wrapper handles it. All other layers apply.
-

## Reference

### Files

| Path                                   | Role                                            |
|----------------------------------------|-------------------------------------------------|
| crates/ragent-tools-core/src/bash.rs   | The BashTool implementation                     |
| crates/ragent-tools-core/src/bg.rs     | Background shell integration                    |
| crates/ragent-config/src/config.rs     | BashConfig struct and defaults                  |
| crates/ragent-config/src/bash_lists.rs | Runtime allowlist/denylist snapshot             |
| crates/ragent-config/src/yolo.rs       | YOLO mode                                       |
| crates/ragent-tui/src/app/slash.rs     | /bash slash commands                            |
| crates/ragent-tui/src/app/init.rs      | load_from_config() at startup                   |
| crates/ragent-types/src/resource.rs    | Process/tool semaphores (16 processes, 5 tools) |

### Config JSON keys (summary)

| Key            | Type     | Default | Meaning                                                |
|----------------|----------|---------|--------------------------------------------------------|
| bash.allowlist | string[] | []      | Command prefixes that bypass the banned-command check  |
| bash.denylist  | string[] | []      | Substring patterns that always reject a command        |
| yolo           | bool     | false   | Master switch that bypasses security layers 2, 3, 6, 7 |

### Examples

**Re-enable curl (without YOLO) but keep it out of rm destructive forms:**

```
{
  "bash": {
    "allowlist": ["curl", "wget"],
    "denylist": ["rm -rf /", "git push --force"]
  }
}
```

---

*This document describes the bash tool as implemented in ragent v1.0.81.*